

Hackathon IA: Desarrollando soluciones con ingeniería Información para la competencia

Contexto y objetivo

Una planta industrial opera un parque de máquinas rotativas (motores eléctricos acoplados a bombas hidráulicas) que son críticas para su proceso productivo. Cada máquina está instrumentada con sensores de vibración, corriente, tensión, temperatura, presión y caudal, cuyas señales se agregan en ventanas temporales para producir descriptores estadísticos y espectrales.

El equipo de mantenimiento confía en un sistema de monitoreo de condición que, a partir de estas mediciones, debe evaluar el estado de salud de cada máquina y decidir qué acción tomar. Las fallas pueden pertenecer a cuatro familias:

- **Mecánicas:** desbalance, desalineación, fallas en rodamientos, lubricación deficiente.
- **Estructurales:** pérdida de rigidez, desajuste de tornillería.
- **Eléctricas:** desequilibrio de tensiones, pérdida de fase, barras rotóricas, cortocircuito entre espiras.
- **Hidráulicas:** cavitación, obstrucción o fuga en el circuito hidráulico.

En total, se consideran 13 modos de falla distintos (F01–F13), que pueden presentarse de forma individual o combinada en una misma ventana de medición. Para detalles específicos de los tipos de falla, ver Tabla 1.

El objetivo es desarrollar un modelo que, a partir de los datos de sensores de una ventana temporal, estime la probabilidad de cada falla y permita al sistema de mantenimiento decidir automáticamente la acción más económica.

Una máquina puede presentar múltiples fallas simultáneamente (estado combinado). El problema es de clasificación *multilabel*: para cada ventana hay 13 probabilidades independientes, una por falla.

Tabla 1 – Detalle de modos de falla

ID	Familia	Descripción
F01	Mecánica	Desbalance de rotor
F02	Mecánica	Desalineación
F03	Mecánica	Rodamiento, pista exterior
F04	Mecánica	Rodamiento, pista interior
F05	Mecánica	Elemento rodante (BSF)
F06	Mecánica	Lubricación deficiente
F07	Estructural	Pérdida de rigidez / tornillería
F08	Eléctrica	Desequilibrio de tensión / fases
F09	Eléctrica	Pérdida de fase / intermitencia
F10	Eléctrica	Barras rotóricas
F11	Eléctrica	Cortocircuito entre espiras
F12	Hidráulica	Cavitación
F13	Hidráulica	Obstrucción / fuga hidráulica

Datos disponibles

El conjunto de datos presenta mediciones de máquinas industriales en distintas condiciones de operación. Se distribuye en formato tabular (CSV y Parquet) y señales crudas (NPZ).

Tabla 2 – Detalle de archivos

Tabla	Descripción	Tiene etiquetas
train_sup	Ventanas etiquetadas para entrenamiento	Sí (label_F01...F13 , severity_F01...F13)
train_unlabeled	Ventanas sin etiquetas, para análisis no supervisado	No
test	Ventanas a predecir. El objetivo del hackathon.	No
raw_signal_index	Índice que mapea window_id → archivo NPZ y canales	No

Identificadores

- **window_id**: identificador único de cada ventana temporal. Es la clave para los *joins* entre tablas y para la entrega.
- **machine_id**: identifica la máquina física. Varias ventanas pueden provenir de la misma máquina.
- **session_id**: identifica la sesión de captura dentro de una máquina.

Importante: al validar localmente, se recomienda separar entrenamiento y validación por **machine_id** para evitar fugas.

Variables tabulares

Cada ventana contiene descriptores agregados por sensor, siguiendo el patrón **<sensor>__<métrica>**. Algunos ejemplos:

- **acc_radial_a__rms**: valor RMS de la vibración radial (acelerómetro A).
- **acc_radial_a__1x**: amplitud espectral a la frecuencia de rotación (1x).
- **acc_radial_a__kurtosis**: curtosis de la señal de vibración.
- **voltage_a__rms**: tensión RMS de la fase A.
- **current_a__rms**: corriente RMS de la fase A.
- **temp_winding__mean**: temperatura media del devanado.
- **pressure_in__mean**: presión media a la entrada de la bomba.
- **flow__mean**: caudal medio.
- **rpm__mean**: velocidad de rotación media.
- **Tamb**: temperatura ambiente.

El diccionario completo de columnas se incluye en **data_dictionary.csv** dentro del dataset.

Señales crudas (NPZ)

Además de los descriptores tabulares, se entregan las señales crudas de tres canales por ventana, en archivos NPZ comprimidos:

- **acc_radial_a**: acelerómetro radial (20 kHz)
- **current_a**: corriente de fase A (10 kHz)
- **pressure_in**: presión de entrada (200 Hz)

Cada archivo `raw_signals/S<hash>.npz` contiene las señales de una sesión completa. Las claves internas siguen el formato `window_id_channel.raw_signal_index.csv` indica, para cada `window_id`, el archivo NPZ correspondiente y los canales disponibles.

El uso de las señales crudas es opcional. Los descriptores tabulares son suficientes para construir un modelo competitivo, pero las señales crudas permiten extraer *features* adicionales más informativos.

Si se prefiere trabajar con CSV longitudinal, usar el script `convert_raw_to_csv.py` incluido en el kit.

Etiquetas y severidad

- `label_F01...label_F13`: binario (0/1), indica si la falla está presente.
- `severity_F01...severity_F13`: valor continuo en [0,1], indica la severidad de la falla cuando está presente (0 = incipiente, 1 = severa).

Métrica de evaluación

La evaluación se basa en el impacto económico de las decisiones de mantenimiento que el sistema toma a partir de las probabilidades predichas. Para cada ventana, el sistema elige automáticamente la acción de menor costo esperado entre cuatro opciones:

Tabla 3 – Decisiones de mantenimiento y costos asociados

Acción	Significado	Costo
MONITOR	No intervenir	\$0 si la máquina está sana; \$5 (leve) o \$15 (severo) si hay falla no detectada
INSPECT familia	Inspeccionar una familia de fallas	\$4 + \$8 si la familia es incorrecta
DERATE	Operar a carga reducida	\$7 + 35% del falso negativo residual
STOP	Detener la máquina	\$12 (sin falso negativo)

El *score* final combina tres componentes ponderadas:

$$final_{score} = 0,7 \cdot cost_{score} + 0,2 \cdot prob_{score} + 0,1 \cdot diag_{score}$$

Costo de decisión (*cost_score*)

Compara el costo medio de las decisiones del sistema contra el costo de no hacer nada (MONITOR siempre):

$$cost_{score} = 100 \cdot \max(0; 1 - mean_cost / naive_cost)$$

Donde

- **100**: las decisiones eliminan todo el costo de no hacer nada.
- **0**: las decisiones son tan malas como no hacer nada.

Calibración probabilística (*prob_score*)

Mide la calidad de las probabilidades mediante el Brier *score*: media de $(p_{pred} - y_{true})^2$ sobre todas las ventanas y etiquetas:

$$prob_{score} = 100 \cdot \max(0; 1 - brier / 0,25)$$

Donde

- **100**: probabilidades perfectas.
- **0**: peor que aleatorio.

Capacidad de diagnóstico (*diag_score*)

Mide la clasificación correcta de fallas con macro-F1 (umbral 0,5):

$$diag_{score} = 100 \cdot macro_f1$$

Donde

- **100**: todas las fallas detectadas sin falsos positivos.
- **0**: ningún acierto.

Interpretación general

- **Métrica = 100**: modelo perfecto. Las decisiones eliminan todo el costo, las probabilidades están perfectamente calibradas y todas las fallas se detectan.
- **Métrica ≈ 25–35**: piso esperado de un *baseline* trivial (asignar la prevalencia observada).
- **Métrica ≈ 45–55**: buen modelo, seguramente con *feature engineering* y calibración.
- **Métrica > 55**: excelente. Difícil de alcanzar; requiere *ensemble*, calibración fina y manejo de fallas raras.
- **Cuanto más alto, mejor.**

El *score* se encuentra implementado dentro de los archivos adjuntos.

Formato de entrega

La entrega es un archivo CSV con una fila por *window_id* del conjunto de *test* y 14 columnas:

window_id, F01, F02, F03, F04, F05, F06, F07, F08, F09, F10, F11, F12, F13

- *window_id*: identificador de la ventana (debe coincidir exactamente con los del archivo *test*).
- F01...F13: probabilidad estimada de cada falla, en el rango [0;1]. Deben ser valores numéricos finitos.

El archivo *baseline_submission.csv* generado por el kit sirve como ejemplo de formato válido.

Kit del participante

Se distribuye un kit autocontenido con todo lo necesario para participar:

Archivo	Descripción
<code>scoring.py</code>	Métrica oficial (cost_score, prob_score, diag_score)
<code>baseline.py</code>	Baseline P0 (prevalencia constante) con <i>holdout</i> agrupado por máquina
<code>convert_raw_to_csv.py</code>	Conversor opcional de señales NPZ a CSV longitudinal
<code>eda_intro.ipynb</code>	Notebook introductorio: estructura de datos, EDA, carga de señales crudas
<code>baseline.ipynb</code>	Notebook pedagógico: construcción del baseline, evaluación y entrega
<code>requirements.txt</code>	Dependencias del kit

Envíos y reglas

1. **Múltiples envíos:** se permiten múltiples envíos durante la competencia. Se tomará como definitivo el último intento realizado.
2. **Prohibido usar el test para entrenar:** el conjunto de test no debe utilizarse para entrenar, etiquetar manualmente, o ajustar hiperparámetros.
3. **Prohibido el etiquetado manual:** no está permitido etiquetar ventanas de test por inspección humana o *feedback* experto.
4. **Señales crudas opcionales:** el uso de señales crudas (NPZ) es opcional. Los descriptores tabulares son suficientes para un modelo competitivo, al menos esa es la promesa que dejó el becario antes de abandonar su puesto en la empresa.
5. **Reproducibilidad:** al finalizar, se deberá adjuntar el código que replica la solución enviada. Para ello se dispondrá de un formulario dedicado.

Para el envío de su propuesta, deberá acceder a "<http://192.168.25.239:8000/>" desde su navegador, iniciar sesión con las credenciales proporcionadas a su grupo vía correo electrónico y subir el "*submit.csv*" con las predicciones de su modelo. En tiempo real podrá ver el desempeño de los demás participantes, accediendo al *ranking* generado en la plataforma. Podrá realizar múltiples envíos, pero **se tomará como definitivo al último intento realizado**.